

Chap 3 : Basics of HTML5

Introduction:

HTML5 is the fifth and current major version of the Hyper Text Markup Language (HTML), the core language for creating web pages and web applications. It introduced new features, elements, and APIs to make web development easier and more powerful. Essentially, HTML5 brought a more semantic and structured approach to web page design, replacing older, more rigid methods.

Key Features of HTML5

HTML5 introduced several new features and improvements. A major goal was to make it easier for web developers to create rich, interactive content without relying on proprietary plugins like Adobe Flash.

- **Semantic Elements:** These are new tags that describe the meaning of the content they contain. Instead of using generic `<div>` tags with `id` or `class` attributes to define sections, you can now use tags like `<header>`, `<nav>`, `<section>`, `<article>`, `<footer>`, and `<aside>`. This makes the code more readable and helps search engines and screen readers understand the structure of the page.
 - **Multimedia Support:** HTML5 made it possible to embed audio and video directly into web pages using the `<audio>` and `<video>` tags. This eliminated the need for third-party plugins. It also introduced the `<canvas>` element for drawing graphics and animations using JavaScript.
 - **New Form Input Types:** HTML5 added new input types like `email`, `url`, `date`, `number`, and `range`. These types provide better validation and a more user-friendly experience on various devices.
 - **Geolocation API:** This API allows a web application to access the user's location (with their permission), enabling location-based services.
 - **Web Storage:** This feature provides a way for web applications to store data locally on the user's machine using `localStorage` and `sessionStorage`. This is an alternative to cookies with a much larger storage capacity.
-

Basic Structure of an HTML5 Page

The basic structure of an HTML5 document is straightforward. All HTML documents must start with a document type declaration, which for HTML5 is a simple `<!DOCTYPE html>`.

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My First HTML5 Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

- **`<!DOCTYPE html>`:** This declaration tells the browser that the document is an HTML5 document.

- **<html>**: This is the root element of the page.
- **<head>**: This section contains metadata about the document, such as the title, character set, and links to stylesheets or scripts.
- **<body>**: This section contains the visible content of the web page, including text, images, and other elements.

Semantic Element:

Semantic elements in HTML5 are tags that give meaning and structure to the content within them. Unlike generic tags like `<div>` and `<span>`, which have no inherent meaning, semantic elements describe the type of content they contain, making the code more readable for both developers and machines. This is a core concept of HTML5 and a significant improvement over older HTML versions.

Why Use Semantic Elements?

Using semantic elements provides several key benefits:

- **Improved Readability:** The code becomes more intuitive and easier for other developers to understand. For example, seeing a `<nav>` tag immediately tells you that the content inside is a navigation menu.
 - **Enhanced Accessibility:** Screen readers and other assistive technologies use these tags to help visually impaired users navigate a webpage. A screen reader can jump directly to the `<main>` content or skip to the `<footer>`, making the user experience much smoother.
 - **Better SEO (Search Engine Optimization):** Search engines like Google use semantic elements to understand the structure and hierarchy of a webpage, which can help them rank the content more effectively in search results. The search engine can identify the main content of the page, the header, and the navigation, improving its ability to index the page correctly.
-

Common Semantic Elements

Here are some of the most widely used semantic elements in HTML5:

1. **<header>**: Represents the introductory content or a group of navigational aids. This is typically found at the top of a page and can contain headings, logos, and navigation links.
2. **<nav>**: Represents a section containing navigation links, such as a main menu or a table of contents.
3. **<main>**: Represents the main content of the `<body>` of a document. There should only be one `<main>` element per page, and it should contain the unique content of the page, not repeating elements like sidebars or footers.
4. **<article>**: Represents a self-contained, independent piece of content that could be distributed and reused independently, such as a blog post, a forum comment, or a news story.
5. **<section>**: Represents a standalone section of a document. It is used to group related content, often with a heading. Examples include chapters, introduction sections, or news topics.
6. **<aside>**: Represents content that is tangentially related to the content around it, often a sidebar. This could include a glossary, related links, or an author bio.
7. **<footer>**: Represents the footer of a document or a section. It typically contains information about the author, copyright data, or links to related documents.

8. **<figure> and <figcaption>:** The `<figure>` element is used to encapsulate content like images, diagrams, or code snippets, while the `<figcaption>` provides a caption for that content.
9. **<mark>** The HTML5 semantic element `<mark>` is used to highlight or mark text that is of particular relevance or importance within a document. The purpose of the `<mark>` tag is to call the user's attention to a specific part of the text, often in the context of a search result or to emphasize a key point.

Unlike other highlighting methods, like using the `<b>` or `<i>` tags, which are for presentational purposes, `<mark>` is **semantic**, meaning it conveys meaning about the content. It indicates that the text is "marked" for a reason.

Example: Highlighting a Search Term

A very common use of the `<mark>` tag is to highlight a user's search query on a results page.

HTML

```
<p>The search for "<strong>HTML5</strong>" returned the following results:</p>
```

```
<ul>
  <li>Learn about the new <mark>HTML5</mark> semantic elements.</li>
  <li>Understand the basics of <mark>HTML5</mark> forms.</li>
  <li>A complete guide to <mark>HTML5</mark> video and audio.</li>
</ul>
```

10. The `<progress>` element has two main attributes:

- **value:** This attribute specifies how much of the task has been completed. The value must be a valid floating-point number between 0 and the maximum value defined by the `max` attribute.
- **max:** This attribute defines the total amount of work required for the task. It represents the value at which the task is considered 100% complete. If not specified, the default value is 1.

The element can be in an **indeterminate state** if the `value` attribute is omitted. In this state, the progress bar doesn't show a specific percentage but indicates that a task is ongoing without a known completion time, such as when a file is being uploaded.

Example

Here's an example demonstrating the use of the `<progress>` element:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Progress Example</title>
</head>
<body>

  <h1>File Upload</h1>
  <label for="file_progress">Uploading your file:</label>
  <progress id="file_progress" value="75" max="100"></progress>
  <p>75% complete</p>

  <h2>Processing Data</h2>
  <p>Please wait, processing data...</p>
  <progress></progress>
```

```
</body>
</html>
```

In this example, the first `<progress>` bar shows a **determinate** progress of 75%, indicating that 75 out of 100 units of work are done. The second `<progress>` bar, without a `value` or `max` attribute, is in an **indeterminate** state, showing that a process is happening without specifying a completion percentage.

11. The `<summary>` element in HTML5 is a semantic element that is used in conjunction with the `<details>` element. It's not a standalone element.

The primary purpose of the `<summary>` tag is to provide a visible heading or title for the content within a `<details>` element. This creates a disclosure widget, a section of content that can be expanded or collapsed by the user.

Think of it as the label on a collapsible menu or an FAQ answer that you can click to reveal more information.

Key points about `<summary>`:

- **Parent Element:** It must be the **first child** of a `<details>` element.
- **Functionality:** Clicking the `<summary>` element toggles the visibility of the content inside the parent `<details>` element.
- **Semantic Value:** It clearly describes to browsers, search engines, and assistive technologies (like screen readers) that the content it holds is a summary or a label for a collapsible section. This is crucial for accessibility and search engine optimization (SEO).

Example

Here's a practical example showing how `<summary>` and `<details>` work together:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>Details and Summary Example</title>
</head>
<body>

  <h1>Frequently Asked Questions</h1>

  <details>
    <summary>What is HTML?</summary>
    <p>HTML, or HyperText Markup Language, is the standard markup language for documents designed to be displayed in a web browser.</p>
  </details>

  <details>
    <summary>What is CSS?</summary>
    <p>CSS, or Cascading Style Sheets, is a style sheet language used for describing the presentation of a document written in a markup language like HTML.</p>
  </details>

</body>
</html>
```

In this example, the user sees "What is HTML?" and "What is CSS?" as two clickable headings. When they click on one, the `<details>` element expands to show the `<p>` tag content. The `<summary>` tag provides the title for each collapsible section.

12.The `<time>` element in HTML5 is used to represent a specific point in time or a duration. It's a semantic element, meaning it provides meaning to the content, which helps browsers and search engines better understand the data. While the element itself doesn't change how the text appears visually, it's crucial for machine-readability and can be used by various applications.

How to Use the `<time>` Element

The `<time>` element typically has two parts: the human-readable content and the machine-readable `datetime` attribute.

- **Content:** This is the part displayed on the webpage and is meant for human readers. It can be a date, a time, or a combination of both.
- **`datetime` attribute:** This is the most important part for machine-readability. It must be in a specific, standardized format (ISO 8601) so that computers can easily parse it.

Example 1: Full Date and Time

To specify a precise moment, you can include the full date and time in the `datetime` attribute.

HTML

```
<p>
  The meeting will be held on <time datetime="2025-10-26T10:00:00Z">October 26th
  at 10:00 AM UTC</time>.
</p>
```

In this example, the user sees "October 26th at 10:00 AM UTC," but the browser and other applications can read the exact, standardized time "2025-10-26T10:00:00Z."

Example 2: Just a Date

You can use the `<time>` element to mark up just a date.

HTML

```
<p>
  My birthday is on <time datetime="1995-12-07">December 7, 1995</time>.
</p>
```

Here, the `datetime` attribute contains only the date in **YYYY-MM-DD** format.

Example 3: Just a Time

Similarly, you can specify just a time.

HTML

```
<p>
  The store opens daily at <time datetime="09:00">9:00 AM</time>.
</p>
```

The `datetime` attribute uses the **HH:MM** format.

Example 4: A Duration

While less common, the `<time>` element can also be used to represent a duration.

HTML

```
<p>
  The movie has a runtime of <time datetime="PT1H45M">1 hour and 45
minutes</time>.
</p>
```

The `datetime` attribute uses the ISO 8601 duration format, where **P** stands for period, **T** for time, **H** for hours, and **M** for minutes.

Why Use It?

The primary purpose of `<time>` is to add **semantic meaning**. This helps in several ways:

- **Accessibility:** Screen readers can announce the time in a more helpful way for visually impaired users.
- **Search Engine Optimization (SEO):** Search engines can better understand and index content related to dates and events.
- **Machine-Readability:** It allows for automatic conversion and scheduling. For instance, an application could automatically add an event to a user's calendar based on the `<time>` element's data.
- **Styling:** Developers can easily select and style time elements using CSS selectors.

Form Element:

1. The `<datalist>` element in HTML5 provides an "autocomplete" feature for form inputs. It's a semantic element used with `<input>` to create a dropdown list of predefined options for the user to choose from. Users can either select an option from the list or type in their own value.
-

How to Use the `<datalist>` Element

To use `<datalist>`, you must link it to an `<input>` element using their respective `id` and `list` attributes.

1. **Create the `<datalist>`:** Use the `<datalist>` tag and give it a unique `id`. Inside, add `<option>` tags for each suggestion. The `value` attribute of the `<option>` tag is what will be displayed in the dropdown.
2. **Link to an `<input>`:** Create an `<input>` tag and add the `list` attribute. The value of this `list` attribute must match the `id` of your `<datalist>`.

Example

HTML

```
<label for="browser">Choose your favorite browser from the list:</label>
<input list="browsers" id="browser" name="browser">

<datalist id="browsers">
  <option value="Chrome">
```

```
<option value="Firefox">
<option value="Safari">
<option value="Edge">
<option value="Opera">
</datalist>
```

In this example:

- The `<input>` has a `list` attribute with the value "browsers."
- The `<datalist>` has an `id` with the value "browsers."

This link tells the `<input>` element to use the options defined within the `<datalist>` with the `id` "browsers." When a user clicks on the input field, the list of options will appear as a dropdown. As they type, the list will filter to show only matching options.

Why Use It?

- **Improved User Experience:** It provides suggestions, saving the user time and effort. It's helpful for fields with a limited but not exhaustive set of possible values, like country names, product categories, or a list of previously entered items.
- **Flexibility:** Unlike a `<select>` element, `<datalist>` allows the user to enter a value that is **not** on the list.
- **Accessibility:** It's a standard HTML feature supported by most modern browsers and accessible to screen readers.

2. The `<keygen>` element was a form element in HTML5 used to generate a public-private key pair for secure client authentication. However, it's **deprecated** and **no longer supported by modern browsers**.

How it Worked

When a user submitted a form containing a `<keygen>` element, the browser would:

1. Generate a unique public-private key pair on the user's machine.
2. Encode the **public key** and send it along with the form data to the server.
3. Store the **private key** locally on the user's computer.

The server could then use the public key to authenticate the user for future interactions, similar to how SSL/TLS certificates work.

Deprecated Example (for historical context only)

HTML

```
<form action="/enroll_certificate" method="post">
  <p>Username: <input type="text" name="username"></p>
  <p>Enroll for a security key:</p>
  <keygen name="security_key">
  <input type="submit" value="Submit">
</form>
```

In this example, upon submission, the browser would generate a key pair and the public key would be sent to the server in a field named "security_key."

Why It Was Deprecated

The `<keygen>` element was removed from the HTML standard primarily because:

- **Lack of Adoption:** It was never widely implemented or supported consistently across all browsers.
- **Security Issues:** The process was not well-defined, leading to security vulnerabilities.
- **Modern Alternatives:** More secure and standardized methods for client authentication, such as the **Web Cryptography API** and **Web Authentication API (WebAuthn)**, have since emerged. These modern APIs offer more robust and flexible ways to handle key generation and authentication within a web browser.

3. The `<output>` element in HTML5 is a semantic element used to display the result of a calculation or a user action. It's meant to show output from a script, such as a calculated value from an `<input>` field.

How to Use the `<output>` Element

The `<output>` element works in conjunction with other form elements, particularly `<input>` with the `type="range"` or `type="number"`. It often has a `for` attribute that links it to the input elements whose values are used in the calculation.

Example: A Simple Calculator

This example shows how to use `<output>` to display the sum of two numbers.

HTML

```
<form oninput="total.value = parseInt(a.value) + parseInt(b.value)">
  <label for="a">Number 1:</label>
  <input type="number" id="a" value="10"> +
  <label for="b">Number 2:</label>
  <input type="number" id="b" value="20"> =
  <output name="total" for="a b">30</output>
</form>
```

In this code:

- The `<form>` element has an `oninput` event handler that calculates the sum of the values from inputs `a` and `b`.
 - The `<output>` element has a `name` attribute of "total" and a `for` attribute with the values "a b".
 - The `for` attribute links the `<output>` to the input elements with IDs `a` and `b`, indicating that the output is related to their values.
 - The `oninput` event updates the value of the `<output>` element, displaying the result to the user without a page reload.
-

Why Use It?

- **Semantic Meaning:** It explicitly communicates that the displayed content is an output or result, which improves accessibility and is useful for search engines.
- **Accessibility:** Screen readers and other assistive technologies can identify the element as a result of a calculation.
- **Clarity:** It makes the code more readable and self-documenting. A developer can quickly understand that the element is used for displaying a result.

Form Input Types:

HTML5 introduced several new input types to collect specific kinds of data from users, improving both user experience and data validation.

1. Color:

```
<input type="color">
```

The `color` input type allows the user to select a color using a color picker. The browser provides a native interface for this, which varies slightly between platforms. The value is a seven-character hexadecimal color code (e.g., `#ff0000`).

Example:

HTML

```
<label for="favcolor">Select your favorite color:</label>
<input type="color" id="favcolor" name="favcolor">
```

2. Date:

```
<input type="date">
```

The `date` input type creates a date picker, allowing the user to select a date (year, month, and day). The value submitted to the server is in the format **YYYY-MM-DD**.

Example:

HTML

```
<label for="start">Start date:</label>
<input type="date" id="start" name="trip-start">
```

3. Datetime:

The `<input type="datetime">` element was a form input type in HTML5 designed to allow users to select a date and time, including the time zone. However, it's **deprecated** and has been removed from the HTML standard. It is no longer supported by modern browsers.

Why It Was Removed

The `datetime` input type was removed due to its complexity and a lack of consistent implementation across browsers. Handling time zones is complicated, and browser developers found it difficult to create a user interface that worked intuitively for everyone.

Modern Alternative: `<input type="datetime-local">`

4. Datetime-local:

`<input type="datetime-local">`

The `datetime-local` input type lets the user select both a date and a time, without including a specific time zone. The browser provides a combined date and time picker. The value is in the format **YYYY-MM-DDThh:mm**.

Example:

HTML

```
<label for="meeting">Meeting time:</label>
<input type="datetime-local" id="meeting" name="meeting-time">
```

Note: The `datetime` input type was deprecated and is no longer part of the HTML5 standard. It was meant to handle a date and time with a time zone.

5. Email:

`<input type="email">`

The `email` input type is designed for email addresses. Browsers on mobile devices often change the keyboard layout to include the "@" symbol. The browser also performs basic validation to ensure the entered text is in a valid email format.

Example:

HTML

```
<label for="email">Enter your email:</label>
<input type="email" id="email" name="user-email">
```

6. Month:

`<input type="month">`

The `month` input type allows the user to select a month and a year. The value is in the format **YYYY-MM**.

Example:

HTML

```
<label for="bdaymonth">Birthday month and year:</label>
<input type="month" id="bdaymonth" name="bday-month">
```

7. Number:

`<input type="number">`

The `number` input type is used for numeric values. It restricts the user to entering only numbers and often includes a spinner control to increment or decrement the value. You can use attributes like `min`, `max`, and `step` to define a valid range and interval.

Example:

HTML

```
<label for="quantity">Quantity (between 1 and 5):</label>
<input type="number" id="quantity" name="quantity" min="1" max="5">
```

8. Range:

```
<input type="range">
```

The `range` input type is used for inputting a numeric value within a specific range. It's displayed as a slider, and the exact value is not as important as the relative value. It's often used with `min`, `max`, and `step` attributes.

Example:

HTML

```
<label for="volume">Volume:</label>
<input type="range" id="volume" name="volume" min="0" max="100">
```

9. Search:

```
<input type="search">
```

The `search` input type is a specialized text field for entering search queries. It behaves like a standard text field but has a few unique features. Modern browsers often add a "clear" button inside the field for quick deletion of the query. Mobile keyboards may also adapt to make searching easier.

Example:

HTML

```
<label for="site-search">Search the site:</label>
<input type="search" id="site-search" name="q">
```

10. Tel:

```
<input type="tel">
```

The `tel` input type is used for telephone numbers. It doesn't enforce a specific number format, but it's useful for several reasons. On mobile devices, it automatically brings up a numeric keypad, making it easier for the user to enter their phone number. It also provides semantic meaning to the input, which can be useful for autofill.

Example:

HTML

```
<label for="phone">Enter your phone number:</label>
<input type="tel" id="phone" name="phone">
```

11. Url:

```
<input type="url">
```

The `url` input type is for entering web addresses. The browser performs basic validation to ensure the entered text is in a valid URL format. On mobile devices, this input type typically modifies the on-screen keyboard to include keys like `/` and `.com` for quicker entry.

Example:

HTML

```
<label for="homepage">Homepage URL:</label>
<input type="url" id="homepage" name="homepage">
```

12. Time:

```
<input type="time">
```

The `time` input type allows the user to select a time of day (hours and minutes). The value submitted to the server is in the format **hh:mm**. The browser provides a native time picker interface.

Example:

HTML

```
<label for="appt">Appointment time:</label>
<input type="time" id="appt" name="appt">
```

13. Week:

```
<input type="week">
```

The `week` input type allows the user to select a specific week and year. The value is in the format **YYYY-Wnn**, where "nn" is the week number (e.g., "2024-W25"). The browser provides a native picker for this.

Example:

HTML

```
<label for="week">Select a week:</label>
<input type="week" id="week" name="week">
```

Form Attributes:

HTML5 introduced several new attributes to enhance the functionality and validation of form elements. These attributes provide greater control over user input and improve the overall user experience.

1. autocomplete

The **autocomplete** attribute specifies whether a form or an input field should have an autocomplete feature enabled. Browsers use this to automatically fill in form data based on past user entries (e.g., name, address, credit card number). It can be set to `on` or `off`. It can also accept a variety of values like `name`, `email`, or `tel` to give hints to the browser about the type of data being entered.

Example:

HTML

```
<form action="/login" method="post" autocomplete="on">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username">
</form>
```

2. autofocus

The **autofocus** attribute automatically places the cursor in a specific form element when the page loads, making it ready for user input immediately. Only one element on a page can have this attribute.

Example:

HTML

```
<label for="username">Username:</label>
<input type="text" id="username" name="username" autofocus>
```

3. form

The **form** attribute allows you to place an input element anywhere on a page, even outside of a `<form>` tag, and still associate it with a specific form. The value of this attribute must be the `id` of the form it belongs to.

Example:

HTML

```
<form action="/submit" id="main-form">
  </form>

<input type="text" form="main-form" name="extra_data" placeholder="This input
is outside the form tag">
```

4. formaction

The **formaction** attribute specifies a different URL for submitting a form. It's used on `<input type="submit">` or `<button>` elements to override the `action` attribute of the parent `<form>`.

Example:

HTML

```
<form action="/submit-default">
  <input type="submit" value="Save" formaction="/submit-draft">
  <input type="submit" value="Publish">
</form>
```

5. formenctype

The **formenctype** attribute defines how form data is encoded when submitted. It is used on `<input type="submit">` or `<button>` and overrides the `enctype` attribute of the parent form. Common values are `application/x-www-form-urlencoded` (default), `multipart/form-data` (for file uploads), and `text/plain`.

Example:

HTML

```
<form action="/upload" method="post" enctype="application/x-www-form-urlencoded">
  <label for="file">Upload a file:</label>
  <input type="file" id="file" name="file-upload">
  <input type="submit" value="Upload File" formenctype="multipart/form-data">
</form>
```

6. formmethod

The **formmethod** attribute specifies the HTTP method for sending form data to the action URL. It is used on `<input type="submit">` or `<button>` and overrides the `method` attribute of the parent form. Values can be `get` or `post`.

Example:

HTML

```
<form action="/submit" method="get">
  <input type="submit" value="Submit via POST" formmethod="post">
  <input type="submit" value="Submit via GET">
</form>
```

7. formnovalidate

The **formnovalidate** attribute is a boolean attribute that, when present on `<input type="submit">` or `<button>`, prevents the form from being validated upon submission. This overrides the `novalidate` attribute of the parent form.

Example:

HTML

```
<form action="/submit">
  <input type="email" name="email">
  <input type="submit" value="Validate and Submit">
  <input type="submit" value="Submit without validation" formnovalidate>
</form>
```

8. formtarget

The **formtarget** attribute specifies where to display the response from submitting the form. It is used on `<input type="submit">` or `<button>` and overrides the `target` attribute of the parent form. Common values are `_self`, `_blank`, `_parent`, or a frame name.

Example:

HTML

```
<form action="/page">
  <input type="submit" value="Open in new window" formtarget="_blank">
</form>
```

9. height and width

The **height** and **width** attributes specify the dimensions of an input field, specifically for `type="image"`.

Example:

HTML

```
<input type="image" src="submit-button.png" alt="Submit" width="100" height="50">
```

10. list

The **list** attribute links an `<input>` element to a `<datalist>` element. The value of **list** must be the **id** of the `<datalist>` you want to associate with the input. This creates an autocomplete/suggestion dropdown.

Example:

HTML

```
<label for="browser">Choose a browser:</label>
<input list="browsers" name="browser" id="browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
</datalist>
```

11. min and max

The **min** and **max** attributes define the minimum and maximum acceptable values for an `<input>` element. They are used with `type="number"`, `range`, `date`, `time`, etc., for basic client-side validation.

Example:

HTML

```
<label for="age">Enter your age (18-99):</label>
<input type="number" id="age" name="age" min="18" max="99">
```

12. multiple

The **multiple** attribute is a boolean attribute that allows the user to select more than one value. It's used on `<input type="email">` and `<input type="file">`.

Example:

HTML

```
<label for="emails">Enter emails (separated by commas):</label>
<input type="email" id="emails" name="emails" multiple>

<label for="files">Select files:</label>
<input type="file" id="files" name="files" multiple>
```

13.pattern

The **pattern** attribute specifies a regular expression (RegExp) that the `<input>` element's value must match to be considered valid. This is used for custom client-side validation.

Example:

HTML

```
<label for="zip">Zip Code (5 digits):</label>
<input type="text" id="zip" name="zip" pattern="[0-9]{5}" title="Five digit
zip code">
```